

[The HULK Programming Language](#) > 7 Functions

7 Functions

HULK also lets you define your own functions (of course!). A program in HULK can have an arbitrary number of functions defined before the final global expression (or expression block).

A function's body is always an expression (or expression block), hence all functions have a return value (and type), that is, the return value (and type) of its body.

7.1 Inline functions

The easiest way to define a function is the inline form. Here's an example:

```
function tan(x) => sin(x) / cos(x);
```

An inline function is defined by an identifier followed by arguments between parenthesis, then the `=>` symbol, and then a simple expression (not an expression block) as body, ending in `;`.

In HULK, all functions must be defined before the final global expression. All these functions live in a single global namespace, hence it is not allowed to repeat function names. Similarly, there are no overloads in HULK (at least in "basic" HULK).

Finally, the body of any function can use other functions, regardless of whether they are defined before or after the corresponding function. Thus, the following is a valid HULK program:

```
function cot(x) => 1 / tan(x);  
function tan(x) => sin(x) / cos(x);  
  
print(tan(PI) ** 2 + cot(PI) ** 2);
```

And of course, inline functions (and any other type of function) can call themselves recursively.

7.2 Full-form functions

Since inline functions only allow for a single expression as body (as complex as

that may be), HULK also allows full-form functions, in which the body is an expression block.

Here's an example of a rather useless function that prints 4 times:

```
function operate(x, y) {  
    print(x + y);  
    print(x - y);  
    print(x * y);  
    print(x / y);  
}
```

Note that the following form is discouraged for stylistic reasons:

```
function id(<args>) => {  
    <...>  
}
```

That is, you should either use the inline form with `=>` and a simple expression, or the full form with `{}` and an expression block.